GRADUATION DEFENSE TEMPLATE 2019

LLM-Driven——Iterative Software Testcase Generation Tool

2351269 黄一和
2352048 夏浩博
2352744 关镜文
2351271 黄弋涵

CONTENT

Part 01
Project Goals

Part 02
Implementation Plan

Part 03
Experiment Results

Part 04
Ablation Study

Part 05
Analysis and Summary

同舟共济

GRADUATION DEFENSE TEMPLATE 2019

GRADUATION DEFENSE TEMPLATE

Part 01
Project Goals

同舟共济

Through **stronger prompt** design and **a white-box iteration mechanism**, can the **improved agent** generate higher-quality test cases on real open-source projects?

Background

Centered around an LLM-driven automated test generation tool, encompassing two types of agents:

- naive agent: Conservative single-round generation, serving as a comparison baseline
- improved agent: The primary method of this project

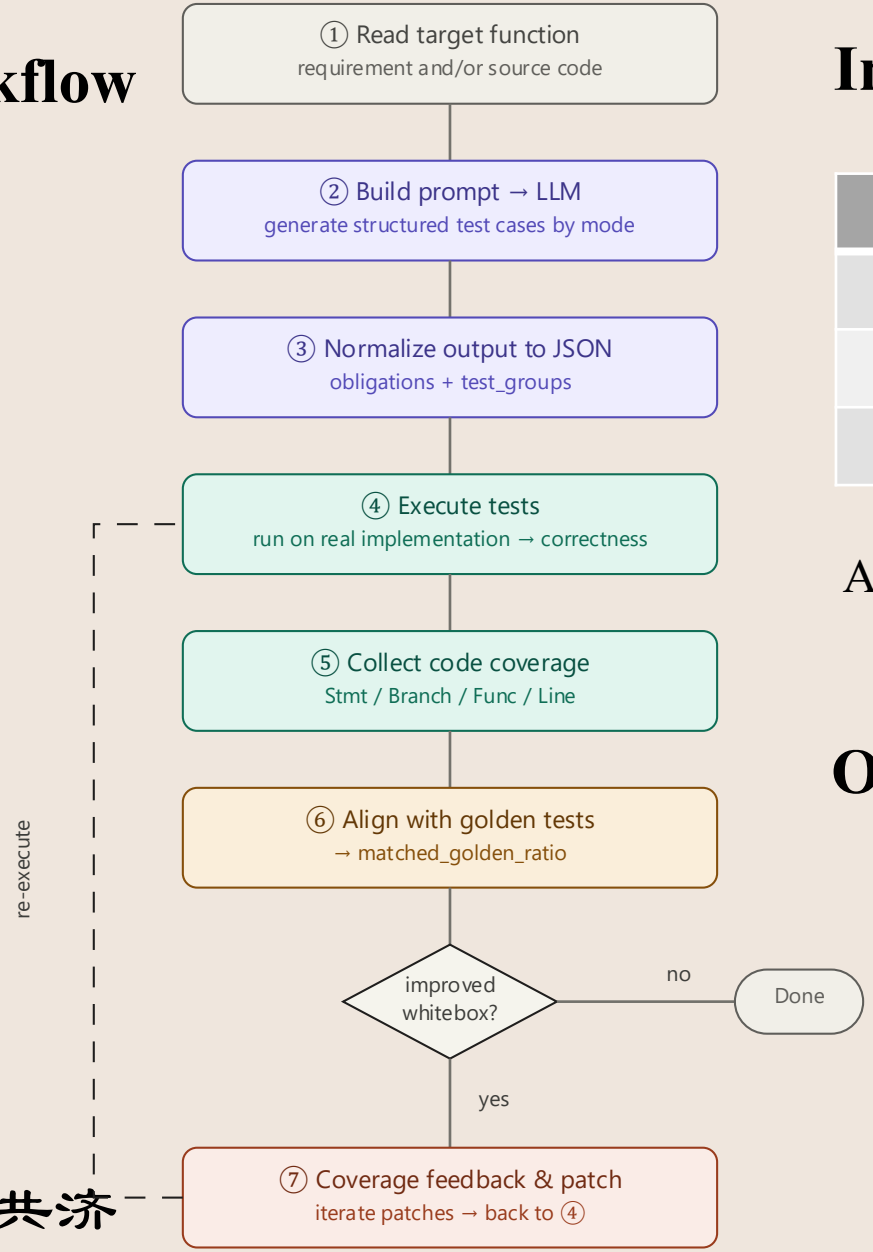
GRADUATION DEFENSE TEMPLATE

Part 02
Implementation Plan

同舟共济

System Design, Input & Output

Workflow



Input:

Mode	Input
blackbox	requirement
whitebox	requirement + source code
whitebox_code_only	source code

Additional Input for Whitebox Iteration: 1. current complete suite
2. uncovered details

Output:

testcases

- Final Output: test suite
- Every Iteration in Whitebox Mode: patch

同舟共济

Naive vs Improved Agent

Naive Agent

A one-round, conservative, low-risk, correctness-oriented baseline generation scheme

Improved Agent

Prompt Enhancement:

- Blackbox: **Obligation enumeration, equivalence partitioning**
- Whitebox/Code-only: **Coverage-guided Iteration**

Whitebox Iteration:

- Per-round input: **Current suite + coverage details + uncovered details**
- Model outputs only "**patch groups**" (incremental), system merges suite + patch

The value of improved white-box: First perform structured initial generation through enhanced white-box prompting, then iteratively patch paths via coverage-guided iteration.

Why Choose validator.js

- Realism: A mature open-source project, not a toy benchmark.
- Completeness: Provides requirements, source code, and manual "golden" test sets.
- Diversity: Functions range from simple validators to those with complex options and branches.

Ground Truth	Purpose
Mature Implementation	Determine if the generated tests execute correctly
Manual Golden Test Set	Evaluate proximity from the perspective of equivalence classes

Three Core Evaluation Metrics

Metric	Meaning	Why it is Needed
Correctness Rate	The proportion of tests that execute correctly on the actual implementation.	Reflects alignment with real requirements. If assertions are wrong, high coverage is meaningless.
Coverage Rate	The four types of coverage: Statement, Branch, Function, and Line.	Evaluates test comprehensiveness at the code level.
Golden Tests Match Rate	The overlap rate of semantic categories compared to expert testing.	Evaluates from a testing design perspective (equivalence partitioning).

Experimental Design

Target Function Set:

- isEmail
- isURL
- isFQDN
- isCurrency
- isCreditCard

Benchmark Selection Principle：

- Avoid overly simple functions→ When all methods perform well, the performance gaps on truly difficult functions become diluted
- Availability of rich "golden tests"

Scale：

- 5 Functions × 3 Modes × 2 Agents = 30 Experiments.
- Modes: blackbox, whitebox, whitebox_code_only
- Agents: naive (baseline), improved (primary method)

Experimental Artifacts：

- Each experimental record captures: correctness, Stmt/Branch/Func/Line coverage, matched_golden_ratio, and token consumption
- Model used: gpt-4o

GRADUATION DEFENSE TEMPLATE

Part 03
Experiment Results

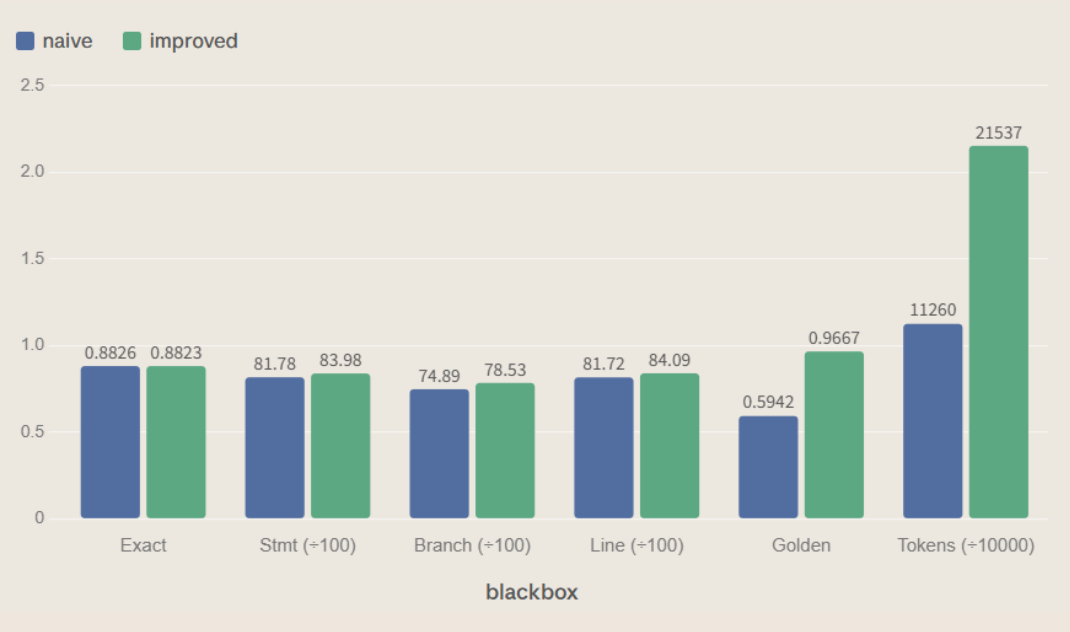
同舟共济

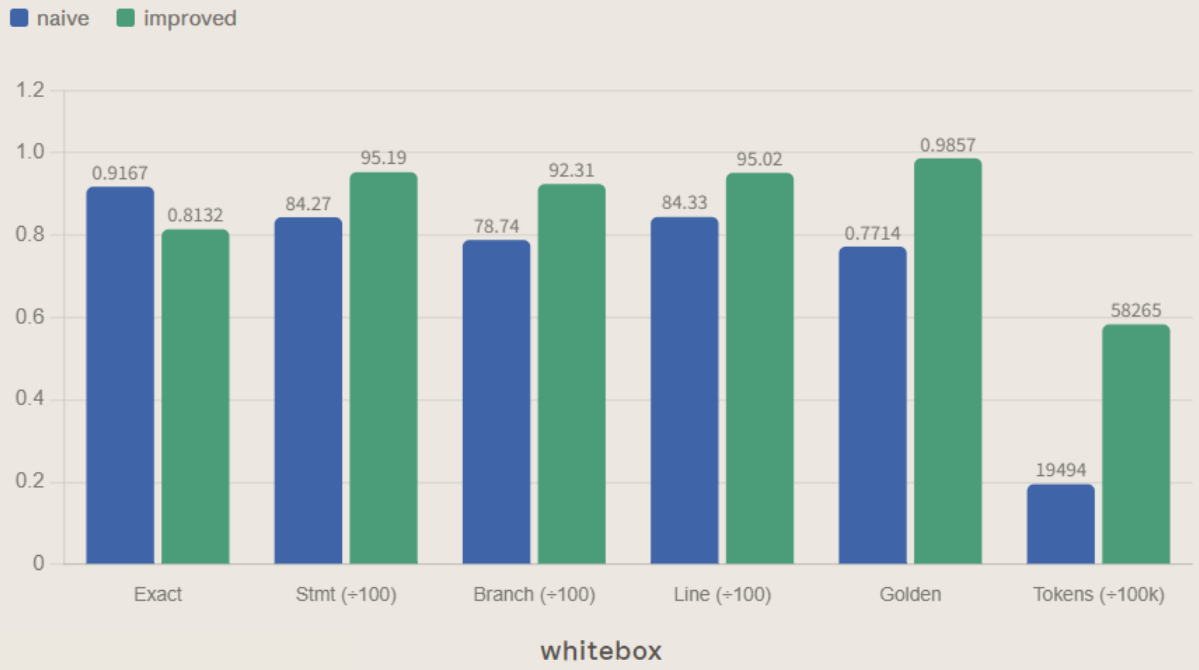
Experiment Result

Agent	Mode	Avg Tokens	Avg Exact Match	Avg Stmt Cov	Avg Branch Cov	Avg Func Cov	Avg Line Cov	Avg Golden Match
`naive`	`blackbox`	11259.80	0.8826	81.78	74.89	100.00	81.72	0.5942
`naive`	`whitebox`	19494.40	0.9167	84.27	78.74	100.00	84.33	0.7714
`naive`	`whitebox_code_only`	18603.80	0.8855	83.71	75.11	100.00	84.49	0.5002
`improved`	`blackbox`	21537.20	0.8823	83.98	78.53	100.00	84.09	0.9667
`improved`	`whitebox`	58264.80	0.8132	95.19	92.31	100.00	95.02	0.9857
`improved`	`whitebox_code_only`	53934.60	0.7793	90.19	81.70	100.00	89.66	0.9312

Detailed comparative analysis

Under the same mode: naive baseline vs improved

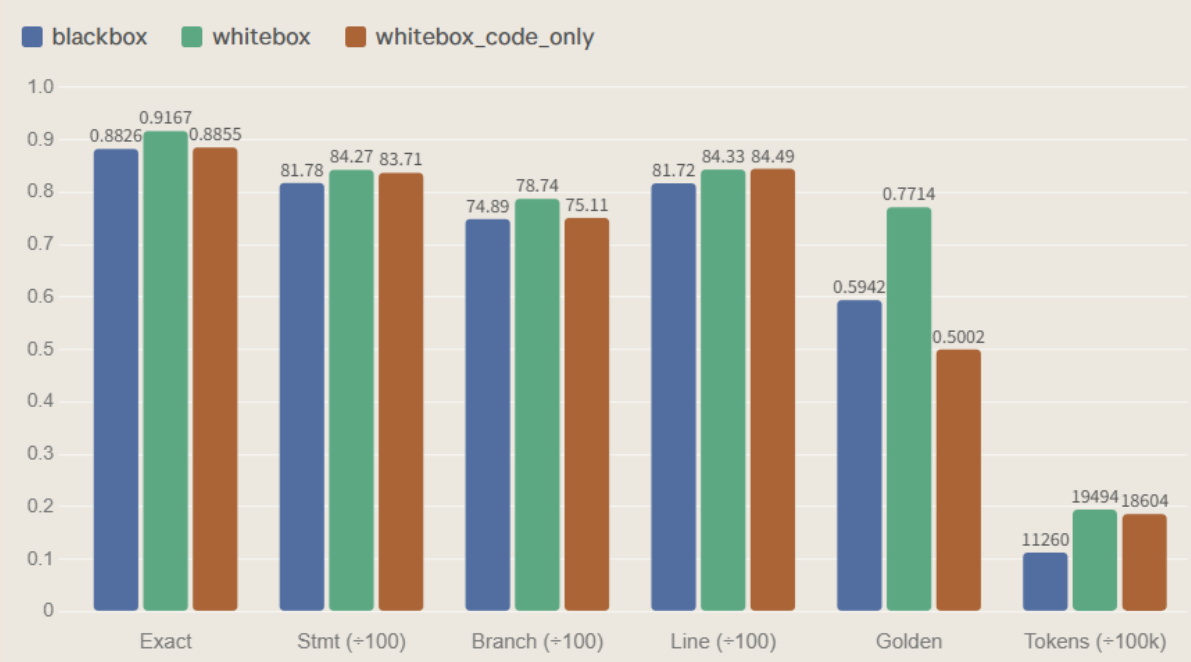




同舟共济

Detailed comparative analysis

Within the same agent: Comparison of different modes



naive — all metrics comparison

improved — all metrics comparison

同舟共济

GRADUATION DEFENSE TEMPLATE

Part 04
Ablation Study

同舟共济

Ablation Study 1: Necessity of the Requirement Re-verification Mechanism

Should the improved whitebox include a requirement re-check?

VERSION	AVG TOKENS	AVG EXACT MATCH	AVG STMT COV	AVG BRANCH COV	AVG FUNC COV	AVG LINE COV	AVG GOLDEN MATCH
`experiment_ablation_1`（requirement re-check）	79361.20	0.8849	88.38	82.59	100.00	88.49	0.7776
`final`（no re-check）	58264.80	0.8132	95.19	92.31	100.00	95.02	0.9857

Analysis：

- Re-checking improves correctness slightly
- But significantly :
  - increases token cost
  - reduces coverage
  - drops golden overlap

Re-checking weakens the two most valuable traits: path exploration and semantic category expansion

Incorrect approach. Deleted

同舟共济

Ablation Study 2: : Iterative ″suite + patch″ vs. Full Rewrite

Why is "suite + patch" better than a pure "suite" rewrite?

VERSION	AVG TOKENS	AVG EXACT MATCH	AVG STMT COV	AVG BRANCH COV	AVG FUNC COV	AVG LINE COV	AVG GOLDEN MATCH
`experiment_ablation_2`	68342.80	0.8164	88.74	81.49	100.00	89.37	0.8430
`final` （suite + patch）	58264.80	0.8132	95.19	92.31	100.00	95.02	0.9857

Analysis：

- suite + patch significantly reduces token costs (68342 → 58264)
- suite + patch significantly improves coverage (Stmt +6.45, Branch +10.82)
- suite + patch significantly improves golden overlap (0.8430 → 0.9857)

suite + patch significantly improves coverage / semantic overlap while reducing costs

同舟共济

GRADUATION DEFENSE TEMPLATE

Part 05
Analysis and Summary

同舟共济

Project Experience

Trivial functions must be excluded

High coverage ≠ High-quality testing

Code-only mode is inherently riskier

Baseline is vital

GRADUATION DEFENSE TEMPLATE 2019

Generalization Ability of the Method

Generalization across function types：

- Tested on 5 functions with distinct styles; stable patterns consistently recur

Generalization across input functions：

- With **requirement** → correctness is more stable
- With **code** → coverage is higher
- With **only code**→ more prone to semantic drift

Generalization across design versions：

- Across multiple rounds of design evolution, core conclusions remain consistent

Boundaries of generalization：

- Current conclusions are reproducible on validator-style functions where code leads to higher coverage
- Generalization to more complex system-level software still requires subsequent verification

同舟共济

GRADUATION DEFENSE TEMPLATE 2019

Comparison with Human Expert Repair

	Human Expert	AI Method
Strengths	Stable correctness, fewer edge-case misreading, high-risk quality control.	Fast generation speed, strong path exploration capability, efficient coverage exploration.
Weaknesses	High cost, difficult to rapidly scale candidate categories.	Potential misunderstanding of requirements, error-prone assertions, semantic drift.

Recommended Workflow:

- Use AI to generate a candidate test suite
- Follow up with manual review or rule-based checks for filtering, correction, and final validation

Core Contributions of “Improved Agent”

1. Enhanced prompt design + coverage-guided patch iteration

2. Sacrificing a portion of accuracy(0~10%) and 1~2 times cost to significantly improve coverage and overlap with golden tests

AI is best suited as a test design accelerator and coverage exploration tool; high-reliability test quality still requires human oversight

GRADUATION DEFENSE TEMPLATE 2019

● Division of Labor

Member	Role	Key Responsibilities
黄一和 (Project Leader)	Overall Design & Reporting	Responsible for project design, experimental scheme design, analysis of results, technical report writing, and project Presentation.
夏浩博	Core Algorithm Implementation	Responsible for the engineering implementation and prompt tuning of the core algorithms (iterative logic of the `improved agent`).
关镜文	Experimental Pipeline	Responsible for the implementation of the automated experimental pipeline based on `validator.js`, including Coverage collection and data interface development.
黄弋涵	Results & Presentation Layout	Responsible for the organization and summarization of experimental results, data statistics, and the design and production of the defense PPT.

同舟共济

GRADUATION DEFENSE TEMPLATE 2019

Thank you!